```
Output:
FINALΣ ΣNON-FINAL
finalς σnon-final
```

Here the special rule is `03A3; 03C2; 03A3; 03A3; Final_Sigma;` which means "U+03A3 Σ lowercases to U+03C2 ς, but only if it is the last letter of a word." Crucially, there is no locale ID in the condition. The normal rule is to simply always use the non-final, lowercase character U+03C3 σ as the lowercase form. As we can see from the output, the special rule is obeyed, and we get the two different lowercase characters depending on their position in the word. We're getting some of the special rules, but not all.

Now let's return to that final clause in the `I` to `ı` mapping rule: "Not_Before_Dot". What's that about? The format of `SpecialCasing.txt` specifies that "The casing context is always the context of the characters in the original string", in other words: the conditions apply to the original character not the result. `0049; 0131; 0049; 0049; tr Not_Before_Dot;` means that the U+0049 character must not come before a dot? Which dot? In this case, it is the U+0307 "COMBINING DOT ABOVE" character, which combines with other codepoints to form a multi-codepoint character (which we're now familiar with thanks to flags and bald men). Our problem case of Turkish characters just got more interesting, because İ could be represented in two ways:

```
$singleCodepointChar = "\u{130}";
$multiCodepointChar = "\u{49}\u{307}";

print "Is it {$singleCodepointChar} or {$multiCodepointChar}?";

Output:
Is it İ or İ?
```

What happens if we feed these two strings into `mb_convert_case()`? Let's find out…

```
$convertedSingle = mb_convert_case($singleCodepointChar, MB_CASE_LOWER);
$convertedMulti = mb_convert_case($multiCodepointChar, MB_CASE_LOWER);

print bin2hex($convertedSingle) . PHP_EOL;
print bin2hex($convertedMulti);

Output:
69cc87
69cc87
```